CT2003_Restaurant_Rush_Group_Assignment_Oral_Presentation.docx Document

How to play this game? What it means by 18 turns? One turn consists what thing and how many people involve? How to start this game? Show me examples of how to play this game?

Restaurant Rush — How to Play the Game

Based on your CT2003 assignment document, **Restaurant Rush is a restaurant management game written in Java**. You play as the restaurant manager. Your job is to serve customers, prepare their food, collect payments, and keep customers satisfied before the game finishes.

The important thing to understand is that **one turn is one round of restaurant operations, not one customer and not one player action**. In each turn, you can make decisions for your staff, but each staff member has a limit on what they can do.

1. What does "18 turns" mean?

The game lasts for a maximum of 18 turns. It must run through all 18 turns, even if you have already met the winning conditions or things are going badly.

Game length

18 turns

Restaurant tables

2 tables

Staff members

3 staff

Scheduled customers

6 customers

Customers arrive at the beginning of turns **1, 4, 7, 10, 13 and 16**.

Think of a turn like a short period of restaurant operation. During that period, your waiter might seat a customer, your chef might prepare food, and your cashier might collect a payment.

How many people are involved?

There are two ways to count the people:

- **You, the manager:** make the decisions.

- **Three staff members:** one Waiter, one Chef and one Cashier.
- **Customers:** six are scheduled to arrive during the game.

So, there is **one manager/player and three staff roles**, plus the customers. The assignment doesn't specify that other human players must take turns controlling the staff; it is a single-manager game.

2. What happens in one turn?

Every turn follows the same six steps, in this order, according to Section 3 of your assignment.

Step 1 — Customers arrive

At the scheduled turns, a new customer joins the waiting queue with satisfaction of 100.

Step 2 — You make staff decisions

- Waiter: perform at most one task, or wait.
- Chef: advance one order by one preparation unit, or wait.
- Cashier: no manual decision is needed; payment is automatic later in the turn.

Step 3 — Kitchen and serving

The Chef prepares food. The Waiter can serve only an order that was already READY when the Waiter acted.

Step 4 — Eating and payment

Customers who were served on an earlier turn finish eating and become eligible to pay. The Cashier automatically collects payment from at most one eligible customer, oldest first.

Step 5 — Waiting customers lose satisfaction

Each waiting or seated customer who has not been served loses 8 satisfaction points. If satisfaction reaches 0, that customer leaves without paying.

Step 6 — Show the results

Display the turn's results, including the number served, unhappy departures, revenue and average satisfaction.

Important: the six steps are one complete turn. You do not start a new turn after every staff member acts.

3. What can you do during a turn?

You control the Waiter and Chef. The Cashier works automatically.



Waiter — choose at most one task per turn

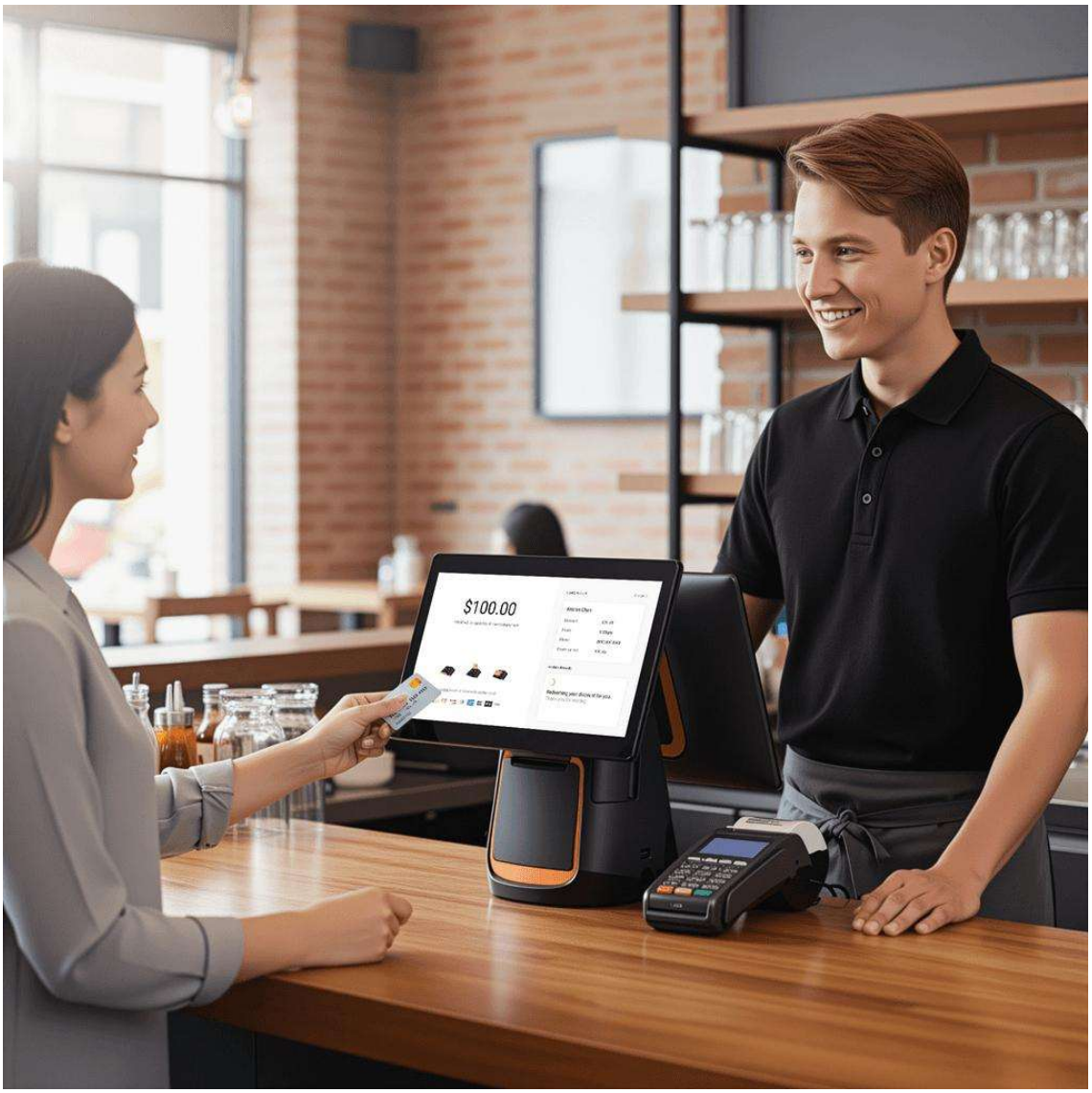
1. Seat one waiting customer at a free table.
2. Take the food order of one seated customer.
3. Serve one customer whose order is already READY.
4. Wait, if you choose not to perform a task.



Chef — choose at most one order to prepare

- Salad: 1 preparation unit.
- Burger: 1 preparation unit.
- Pasta: 2 preparation units.

One preparation unit is one unit of kitchen work. For Pasta, the Chef needs to work on the order over two turns. The Chef can start preparing an order taken by the Waiter in the same turn.



Cashier — automatic payment

The Cashier collects payment from at most one customer who is READY_TO_PAY each turn. You do not need to choose whom to charge; the oldest eligible customer is handled first.

The two important restrictions are:

- The Waiter cannot seat a customer and take an order in the same turn. Both count as separate tasks.
- The Chef and Waiter can both work in the same turn, because they are different staff members.

For example, in one turn, the Waiter can take Customer A's order while the Chef prepares Customer A's food. This is allowed.

4. How do you start the game?

The assignment requires a Java command-line interface (CLI). The exact menu and input format depend on how your group implements the program.

Here is an example of how your game could begin:

Example game screen — illustrative, not actual program output

============================== RESTAURANT RUSH ============================== Welcome, Restaurant Manager! Turn: 1 / 18 Revenue: $0.00 Average satisfaction: N/A Waiting queue: None Occupied tables: 0 / 2 Waiter: 1 task available Chef: 1 preparation unit available Cashier: Automatic A new customer has arrived! Customer 1 Satisfaction: 100 Status: WAITING Choose the Waiter action: 1. Seat a customer 2. Take an order 3. Serve a READY order 4. Wait Enter choice: _

You would play as follows:

1. Run your group's Java program in your IDE or terminal.
2. Read the current turn, revenue, customers, occupied tables and staff availability.
3. Choose a Waiter action. For example, enter 1 to seat a waiting customer.
4. Choose a Chef action. For example, select an available order to prepare, or wait.
5. Let the program process eating, payment, satisfaction loss and departures.
6. Review the results and repeat until the end of turn 18.

You do not choose the customer's food. The customer chooses it according to the deterministic rule or seeded randomness your group implements. The game should display the food choice when the order is placed.

5. Example: how to play one customer from beginning to payment

Let's follow the exact Pasta example in Section 3.3 of your assignment. Imagine Customer 1 arrives and chooses Pasta, which costs $18.00 and requires two preparation units.

Turn 1

Seat the customer

- Customer 1 arrives with satisfaction 100.
- Waiter seats Customer 1 at Table 1.
- Chef waits because there is no order yet.

- Customer is not served, so satisfaction falls to 92.

End satisfaction: 92

Turn 2

Take the order and start cooking

- Waiter takes Customer 1's order.
- Customer chooses Pasta ($18.00).
- Chef completes preparation unit 1 of 2.
- Customer is still waiting for food, so satisfaction falls to 84.

End satisfaction: 84

Turn 3

Finish preparing the Pasta

- Waiter could choose another task, such as seating another customer if a table is free.
- Chef completes preparation unit 2 of 2.
- The Pasta becomes READY.
- The Waiter cannot serve it this turn because it became READY after the Waiter's action.
- Satisfaction falls to 76.

End satisfaction: 76

Turn 4

Serve the food

- Waiter serves Customer 1's READY Pasta.
- Order status changes to SERVED.
- Satisfaction stays at 76 because served customers no longer lose satisfaction while waiting.

Status: SERVED

Turn 5

Finish eating

- Customer 1 completes one full turn of eating.

- Status changes to READY_TO_PAY.
- Payment cannot be collected until the next turn at the earliest.

Status: READY_TO_PAY

Turn 6

Collect payment

- Cashier automatically collects $18.00.
- Customer and order status change to PAID.
- Revenue increases from $0.00 to $18.00.
- Table 1 becomes free.
- Satisfaction of 76 is recorded for the final average.

Paid: $18.00

This example shows why you must plan ahead. One customer ordering Pasta occupies a table for several turns. While that customer is being served, you should try to use the other table and available staff efficiently.

6. Example: what should you do when several customers arrive?

Here is a possible situation during the game. The following decisions are illustrative; customer food choices depend on your group's implemented rule.

Turn 4 — Sample decision

Suppose Customer 1 is seated at Table 1 and has a Pasta order being prepared. Customer 2 has just arrived.



Your Waiter choice

Seat Customer 2 at Table 2, rather than taking another order or serving food that is not READY.



Your Chef choice

Finish Customer 1's Pasta if it still needs its final preparation unit.



Your Cashier

The Cashier automatically collects payment from an eligible customer, if one exists.

Why is this a useful decision?

You are preparing one customer's food while seating another customer. However, both customers who have not yet been served lose satisfaction at the end of the turn. Your challenge is to balance getting more customers into the restaurant against serving customers quickly.

7. What do you need to achieve to win?

At the end of turn 18, **all three conditions must be met**.

At least 4 paid customers

Customers who have merely ordered or eaten do not count as paid.

Revenue of at least $45.00

Only actual payments count. Unpaid and abandoned orders contribute nothing.

Average satisfaction of at least 60

Average the satisfaction scores recorded when customers pay, not their satisfaction at arrival.

For example, suppose five customers have paid:

Customer	Bill	Satisfaction at payment
1	$18.00	76
2	$15.00	84
3	$9.00	68
4	$15.00	72
5	$9.00	80
Total / average	$66.00	76

This would be a **victory**: five paid customers, $66.00 revenue and average satisfaction of 76. All three conditions are satisfied.

One important rule: the customer who arrives on turn 16 cannot finish paying within turn 18, so at most five customers can pay. You need to get enough of the earlier arrivals through the full process.

8. Try a practice round

Use this small scenario to check whether you understand the rules.

Question 1

Customer A is seated, but has not ordered food. What can the Waiter do this turn?

Seat another customer and take Customer A's order

Take Customer A's order

Prepare Customer A's food

Question 2

The Chef finishes the last preparation unit of a dish this turn. Can the Waiter serve that dish in the same turn?

Yes, immediately

No, it can be served next turn at the earliest

Question 3

Customer B has just been served this turn. Can the Cashier collect payment from Customer B in this same turn?

Yes

No, Customer B must complete a full turn of eating first

The main idea is that Restaurant Rush is about **managing limited staff actions across 18 turns**. You must decide when to seat customers, when to take orders, which food to prepare first and when to serve READY dishes. Good planning helps you collect enough payments while keeping satisfaction high.

Check my Restaurant Rush practice quiz answers and explain each one. My answers are: Q1=b, Q2=b, Q3=b. Use the CT2003 assignment rules.

Great! 🎉 **You got all 3 questions correct — 3/3!**

Your answers and explanations

Question 1: B — Take Customer A's order

Correct

The Waiter can perform **at most one task per turn**. Taking Customer A's order uses that task, so the Waiter cannot also seat another customer during the same turn.

Preparing food is the Chef's responsibility, not the Waiter's.

Question 2: B — Serve next turn at the earliest

Correct

The Waiter acts before the Chef. If the Chef finishes the final preparation unit later in the same turn, the dish becomes READY only after the Waiter's action.

Therefore, the Waiter cannot serve it until a later turn.

Question 3: B — Complete one full turn of eating first

Correct

A customer served during the current turn must complete one full turn of eating before becoming READY_TO_PAY.

The Cashier can collect payment on the following turn at the earliest. Payment is automatic, and the Cashier handles at most one eligible customer per turn.

Remember these three rules

Staff / process	Key rule
Waiter	At most one task per turn
Chef	One preparation unit per turn
Cashier	Automatic payment, at most one eligible customer per turn

The most important timing sequence to remember is:

PREPARING — Chef works on the order

READY — Food preparation completed

SERVED — Waiter delivers the food

READY_TO_PAY — Customer finishes eating

PAID — Cashier collects payment

You understand the basic turn rules. The next challenge is deciding which customer to prioritize when both tables are occupied, multiple orders are waiting, and customer satisfaction is dropping.

show me the steps that I can victory for this game?

Restaurant Rush: Step-by-Step Strategy to Win

To win your CT2003 Restaurant Rush game, you need to meet **all three targets by the end of Turn 18**:

Paid customers

At least 4 customers must pay.

Revenue

At least $45.00 in actual payments.

Average satisfaction

At least 60 among customers who paid.

The best strategy is to **get customers seated, take their orders, prepare their food and serve them as early as possible**. Do not let customers wait too long, because every turn they remain unserved costs them 8 satisfaction points.

I'll show you a turn-by-turn example using the assignment's Stage 1 rules. The example assumes customers choose the dishes shown and that the manager makes the suggested decisions.

1. Understand the order of actions

Every turn, remember this sequence:

1. New customer arrives if it is a scheduled arrival turn.
2. You choose one Waiter task.
3. You choose one Chef preparation task.
4. The game processes serving, eating and automatic payment.
5. Unserved waiting or seated customers lose satisfaction.
6. The game displays the turn results.

The Waiter must act before the Chef. A dish that becomes READY during the current turn cannot be served until a later turn.

2. Your winning plan: Turns 1–18

This example uses one-unit dishes (Salad or Burger) so that each order can be prepared in one turn. It assumes the customers choose those dishes; **you cannot force them to choose a particular dish**.

Legend: Seat = seat customer; Order = take order; Prep = Chef prepares; Serve = deliver food; Pay = automatic payment.

Turn	Waiter action	Chef action	Cashier / important event
1	Seat Customer 1	Wait	Customer 1 arrives
2	Order Customer 1	Prepare C1	C1's order becomes READY
3	Serve C1	Wait	C1 starts the served stage
4	Seat Customer 2	Wait	C1 finishes eating; becomes READY_TO_PAY

5	Order C2	Prepare C2	C1 pays
6	Serve C2	Wait	C2 is served
7	Seat Customer 3	Wait	C2 finishes eating
8	Order C3	Prepare C3	C2 pays
9	Serve C3	Wait	C3 is served
10	Seat Customer 4	Wait	C3 finishes eating
11	Order C4	Prepare C4	C3 pays
12	Serve C4	Wait	C4 is served
13	Seat Customer 5	Wait	C4 finishes eating
14	Order C5	Prepare C5	C4 pays
15	Serve C5	Wait	C5 is served
16	Seat Customer 6	Wait	C5 finishes eating
17	Order C6	Prepare C6	C5 pays
18	Serve C6	Wait	Final turn; C6 has not paid yet

Why this schedule works:

- The two tables allow you to seat the next customer while another customer is still being served or waiting to pay.
- The Chef prepares an order on the same turn the Waiter takes it.
- You serve food on the next turn, not in the turn it becomes READY.
- The Cashier handles payments automatically.
- The first five customers have time to pay by turn 18 if the schedule is followed.

3. Check whether you have won

Let's calculate a sample victory if the first five customers choose Burger, priced at $15 each, and each finishes with satisfaction 84.

Customers paid

5 / 4 required

Revenue

$75.00 / $45.00

Average satisfaction

84 / 60

Victory!

All three targets are satisfied.

This is an illustration, not a guaranteed result. Customers choose their own dishes, so revenue depends on the actual orders. For example, five Salads would generate exactly $45.00, whereas four Salads would generate only $36.00.

4. Five tips to improve your chance of victory

1. **Avoid wasting Waiter turns.** Prioritize customers who can be seated, ordered for or served immediately.
2. **Prepare food as soon as an order is available.** A delay in the kitchen can mean more lost satisfaction.
3. **Use both tables.** Seat the next customer as soon as a table is free, even if the other customer is still eating or waiting to pay.
4. **Watch the READY status.** A completed dish cannot be served until a later turn, so plan the Waiter's next action accordingly.
5. **Check all three victory targets.** Revenue alone is not enough; you also need four paid customers and an average satisfaction of at least 60.

One final warning: Pasta takes two preparation units. If a customer orders Pasta, the Chef needs an additional turn to finish it. Adjust the schedule instead of assuming every dish is ready after one preparation turn.

For your assignment, I recommend testing this schedule in your group's Java program and recording the actual customer choices, payments and satisfaction scores. That will give you a reproducible victory run for your test notes and oral presentation.

For this group assignment, I, the manager choose what food for customer to eat?

No. According to your **CT2003 Restaurant Rush assignment, you do not choose the food for the customer.**

Section 2.2 of your assignment explicitly says:

Customers choose their own food. The manager never chooses it.

So, what do you control as the manager?

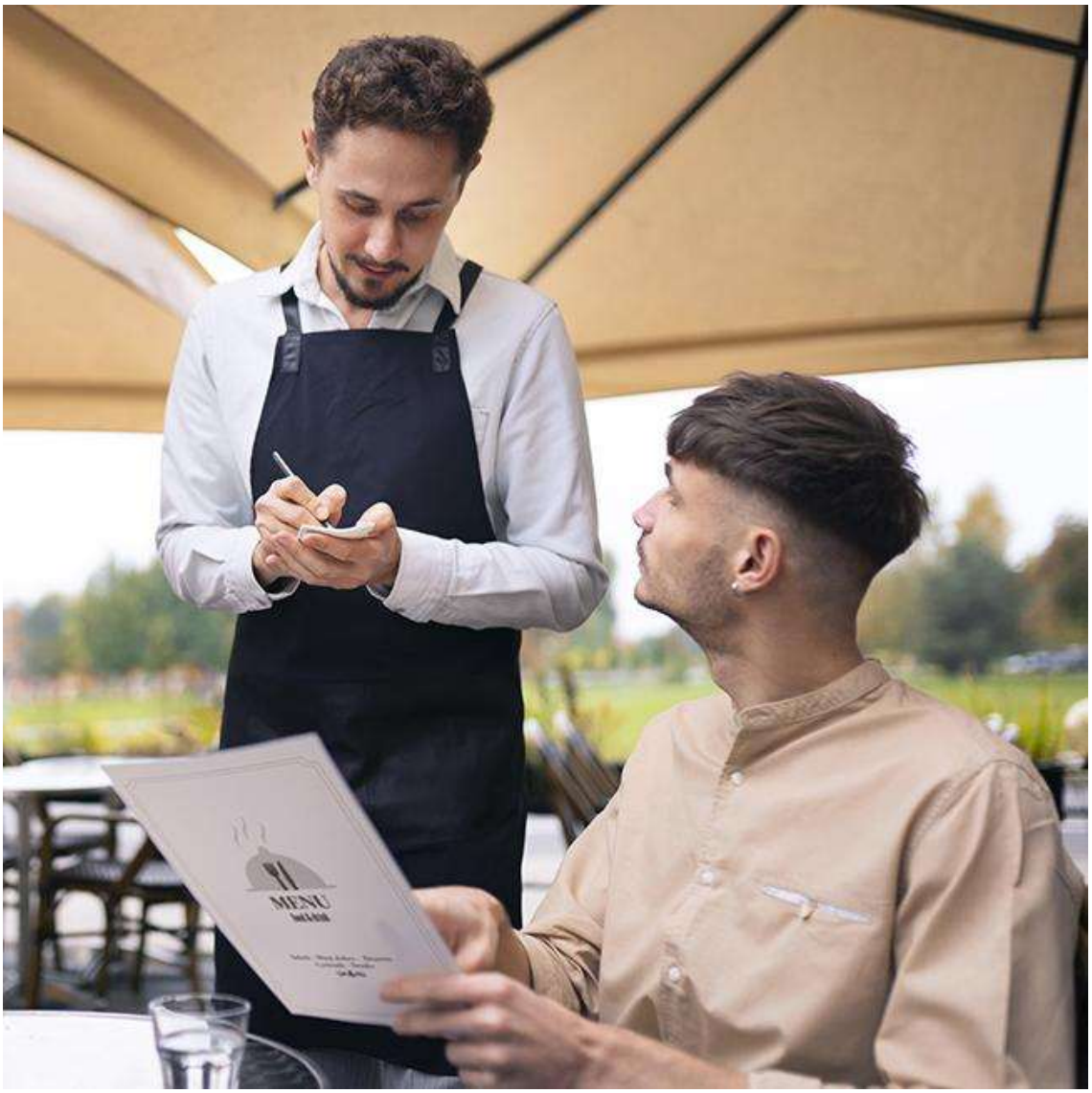
Action	**Who decides or does it?**
Choose Salad, Burger or Pasta	Customer
Seat a customer at a table	You, through the Waiter
Take the customer's order	You, through the Waiter
Prepare the food	You, through the Chef
Serve the READY dish	You, through the Waiter
Collect payment	Cashier automatically

Example of how it works



Step 1: Customer arrives

Customer 1 arrives and you use the Waiter to seat them.



Step 2: Customer chooses food

You select the Waiter action to take the order. The customer might choose Pasta. Your game should display that choice.



Step 3: You manage the preparation

You choose which order the Chef prepares. You are deciding the kitchen priority, not the customer's food choice.

Important for your Java implementation: Your code should contain a documented deterministic rule or seeded randomness for customer food selection. For example, Customer 1 might choose Salad, Customer 2 Burger, and Customer 3 Pasta. These are just example choices; your program determines them according to its implemented rule.

Your role is to manage staff time efficiently so customers receive their chosen food, pay their bills and remain satisfied.

In assignment, what it means "You build the game in three stages, and each stage must show how your object-oriented design supports the next set of requirements"?

It means you should **develop the Restaurant Rush game in three stages**, adding new features at each stage. Your lecturer wants to see that your Java classes and objects are designed in a way that makes adding those new features easier.

Let me explain using your assignment.

1. What are the three stages?

1

Stage 1 — Basic game

Build the basic restaurant simulation with customers, tables, a Waiter, a Chef, a Cashier, and the menu.

Goal: Make the basic game work according to the rules.

2

Stage 2 — Add special customers

Introduce VIP customers and Critics, with different satisfaction loss rates, discounts, or reactions.

Goal: Extend your existing customer design instead of rewriting the whole game.

3

Stage 3 — Add more game features

Introduce a Host, ComboMeal, and Happy Hour.

Goal: Add new functionality while keeping your existing code organized and reusable.

2. What does "object-oriented design supports the next set of requirements" mean?

It means your program should be designed using **classes, objects, inheritance, and other object-oriented concepts** so that new features can be added without changing too much existing code.

For example, imagine you create a Customer class in Stage 1.

```
class Customer {
  protected int satisfaction;

  public void loseSatisfaction(int amount) {
    satisfaction -= amount;
```

```
      if (satisfaction < 0) {
        satisfaction = 0;
      }
    }
}
```

In Stage 1, regular customers lose 8 satisfaction points per turn.

In Stage 2, VIP customers lose only 5 points, while Critics lose 12 points. Instead of writing completely separate customer systems, you could use **inheritance**:

```
class VIPCustomer extends Customer {
  // VIP-specific features
}


class Critic extends Customer {
  // Critic-specific features
}
```

You could then override a method or use a customer-specific value to handle their different satisfaction loss rates.

The important idea is that your original Customer class provides a foundation for the new customer types.

3. Another example: adding a Host in Stage 3

In Stage 1, the Waiter seats customers. In Stage 3, the Host can also seat a customer each turn.

If you organize your program well, you can add a Host class with its own seating responsibility and update the turn logic to support it, without rewriting the entire game.

That is what the assignment means by designing your objects to support future requirements.

4. What your lecturer wants to see

What to demonstrate	**What it means**

Encapsulation	Keep data and related methods inside appropriate classes.
Inheritance	Reuse a base class for specialized types, such as VIP customers.
Polymorphism	Allow different customer types to behave differently through a common interface or parent class.
Extensibility	Add the Host, ComboMeal, and Happy Hour without rebuilding the entire program.
Working stages	Ensure each stage works before extending it with the next stage's requirements.

These are examples of how to demonstrate good object-oriented design; your assignment's exact marking criteria determine which concepts are required.

In simple words: Don't just make the game work. Design your Java classes so that when Stage 2 and Stage 3 introduce new features, you can extend your existing program instead of starting over.